Procedimentos

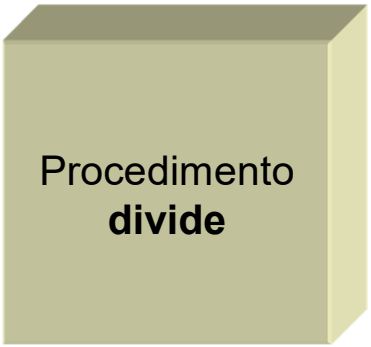
- Em algumas situações desejamos especificar uma operação que não é convenientemente determinada como parte de uma expressão
- Nesses casos, utilizamos outra forma de sub-algoritmo: o procedimento
- Embora a função e o procedimento sejam similares, existem duas diferenças importantes:
  - Numa chamada de procedimento, a execução do programa que o chamou é interrompida, passando o controle ao procedimento chamado. Após a execução do procedimento, o controle retorna ao programa que efetuou a chamada no comando **imediatamente subsequente**. A execução do programa continua a partir desse ponto
  - Não existe retorno de um único valor como no caso da função. Qualquer valor a ser retornado por um procedimento retorna ao programa que efetuou a chamada por meio de seus parâmetros

Procedimento: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor



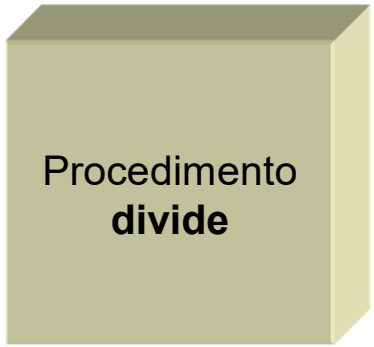
Instruções definindo o procedimento **divide**

Procedimento: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	
y	



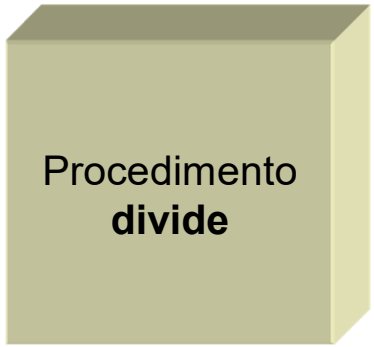
Instruções definindo o procedimento **divide**

Procedimento: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	3
y	



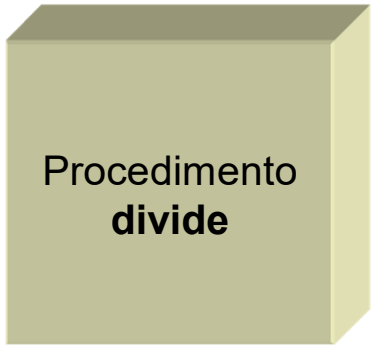
Instruções definindo o procedimento **divide**

Procedimento: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	3
y	2



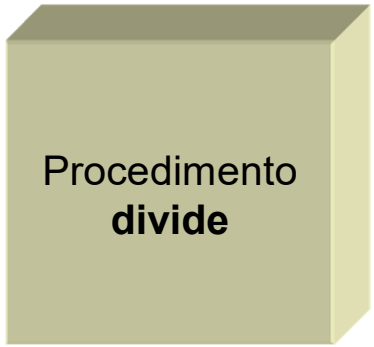
Instruções definindo o procedimento **divide**

Procedimento: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	3
y	2



Instruções definindo o procedimento **divide**

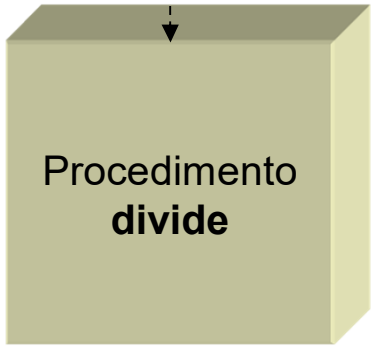
Procedimento: Fluxo de Controle

O controle é transferido para o procedimento **divide**

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	3
y	2



Instruções definindo o procedimento **divide**

Procedimento: Fluxo de Controle

O controle é transferido para o procedimento **divide**

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	3
y	2

Procedimento **divide**

Instruções definindo o procedimento **divide**

O código que define o procedimento é executado, respeitando as estruturas de controle

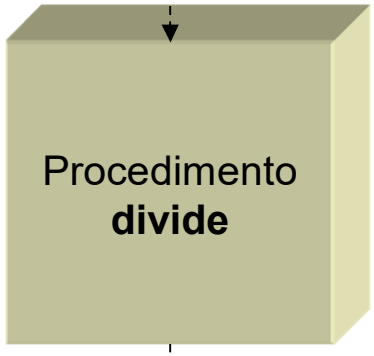
Procedimento: Fluxo de Controle

O controle é transferido para o procedimento **divide**

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor
x	3
y	2



Instruções definindo o procedimento **divide**

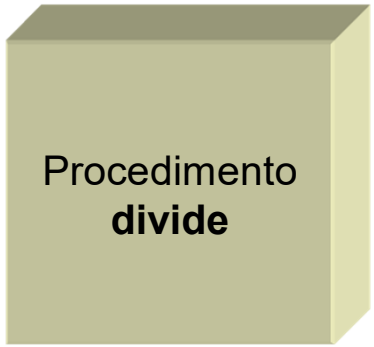
Retorno do controle no comando subsequente

Procedimento: Fluxo de Controle

```
Algoritmo Exemplo
Início
  declare x,y : real

  x ← 3
  y ← 2
  divide(x,y)
  Escreva(x,y)
Fim
```

Memória	
Endereço	Valor



Instruções definindo o procedimento **divide**

Exemplo

```
//-------------------------------------------------------
// Calcula e imprime quociente e resto entre dois inteiros
Procedimento divide(dividendo, divisor : inteiro)
  declare quociente,resto : inteiro

  quociente ← dividendo / divisor
  resto ← dividendo - quociente * divisor
  Escreva("quociente = ",quociente," resto = ",resto)
Fim Procedimento
//-------------------------------------------------------
Algoritmo TestaDivide
Início
  declare a,b : inteiro

  a ← 5
  b ← 3
  divide(a,b)
  divide(a*b-1,b+1)
Fim
```

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Indica um procedimento. Em C/C++ todo procedimento pode ser visto como uma função que não retorna valor algum

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Indica o nome do procedimento.
Neste caso, **divide**

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Indica o tipo do primeiro parâmetro. Neste caso, é um inteiro.

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Indica o nome do primeiro parâmetro. Neste caso, **dividendo**

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Indica o tipo do segundo parâmetro. Neste caso, é um inteiro.

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Indica o nome do segundo parâmetro. Neste caso, **divisor**

Procedimentos em C++

```
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
```

Declara duas variáveis locais do tipo inteiro.

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor
a	5
b	3

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor

Memória	
Endereço	Valor
a	5
b	3

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	5
divisor	3

Memória	
Endereço	Valor
a	5
b	3

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	5
divisor	3
quociente	
resto	

Memória	
Endereço	Valor
a	5
b	3

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	5
divisor	3
quociente	1
resto	

Memória	
Endereço	Valor
a	5
b	3

Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	5
divisor	3
quociente	1
resto	2

Memória	
Endereço	Valor
a	5
b	3

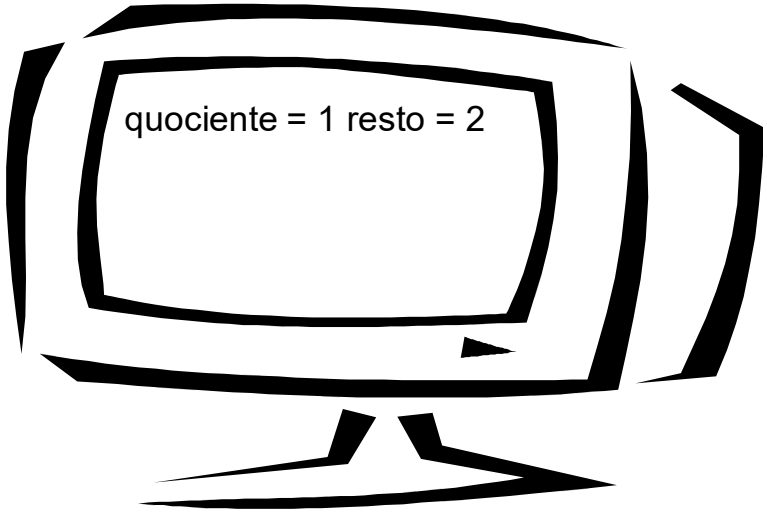
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	5
divisor	3
quociente	1
resto	2



Memória	
Endereço	Valor
a	5
b	3

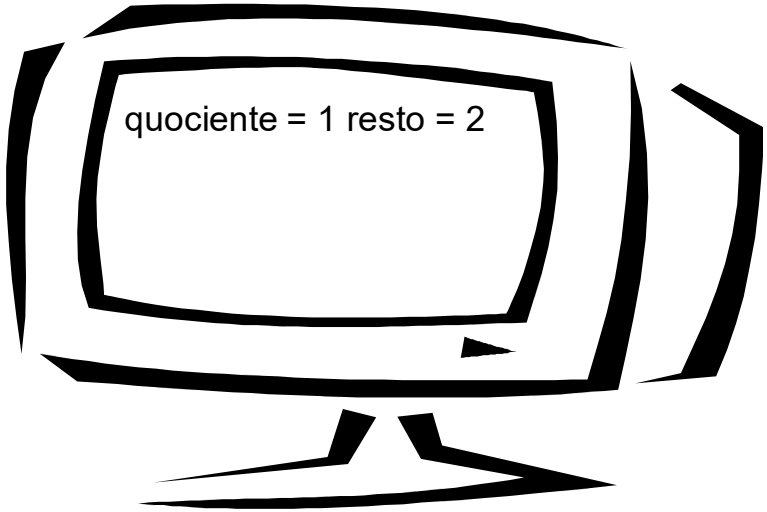
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor



Memória	
Endereço	Valor
a	5
b	3

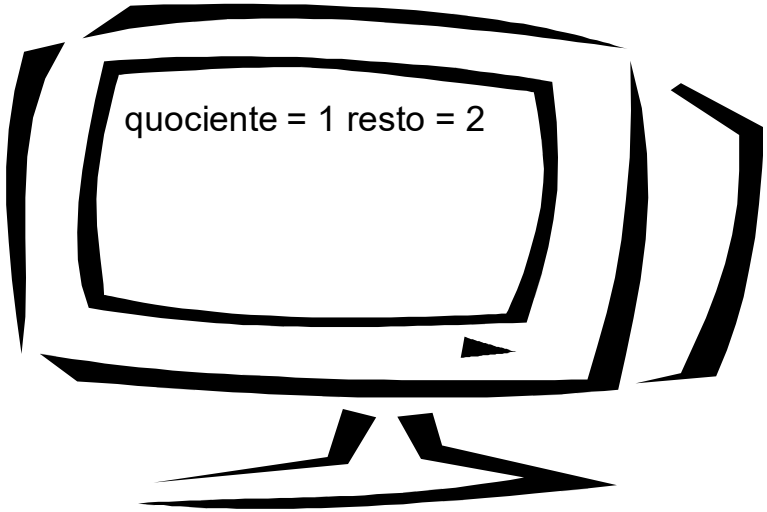
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor



Memória	
Endereço	Valor
a	5
b	3

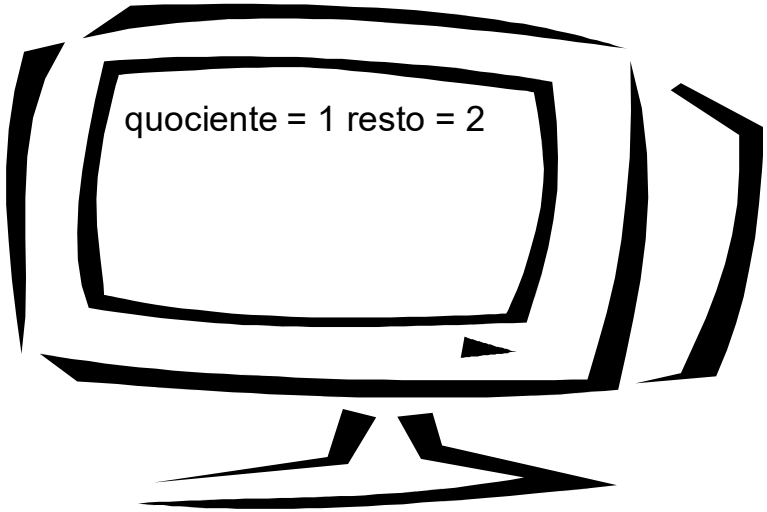
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	14
divisor	4



Memória	
Endereço	Valor
a	5
b	3

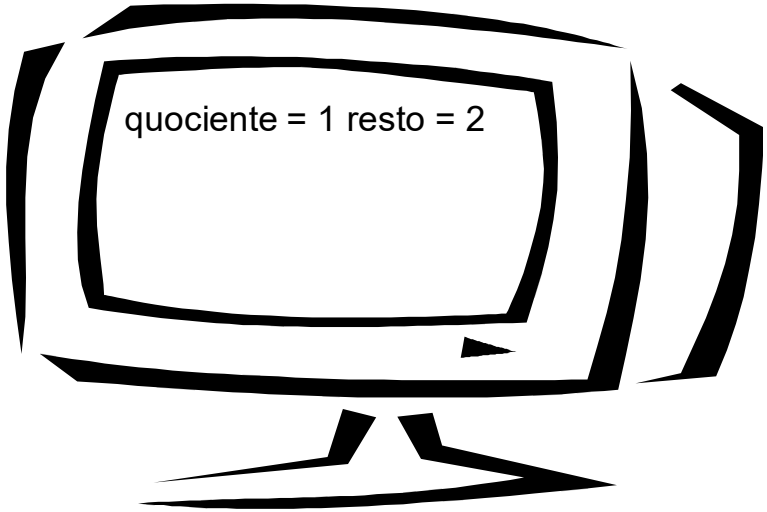
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	14
divisor	4
quociente	
resto	



Memória	
Endereço	Valor
a	5
b	3

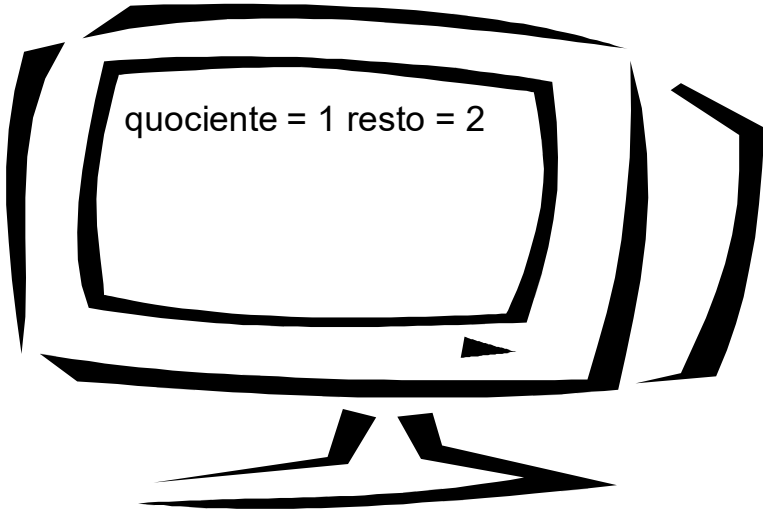
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	14
divisor	4
quociente	3
resto	



Memória	
Endereço	Valor
a	5
b	3

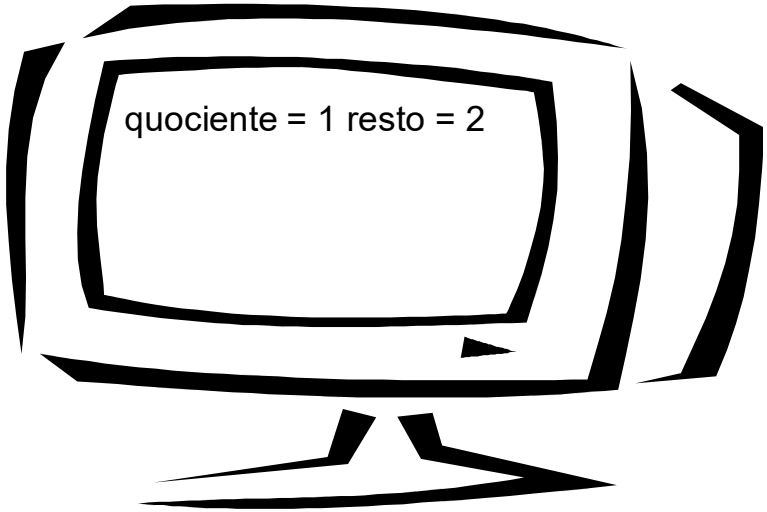
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	14
divisor	4
quociente	3
resto	2



Memória	
Endereço	Valor
a	5
b	3

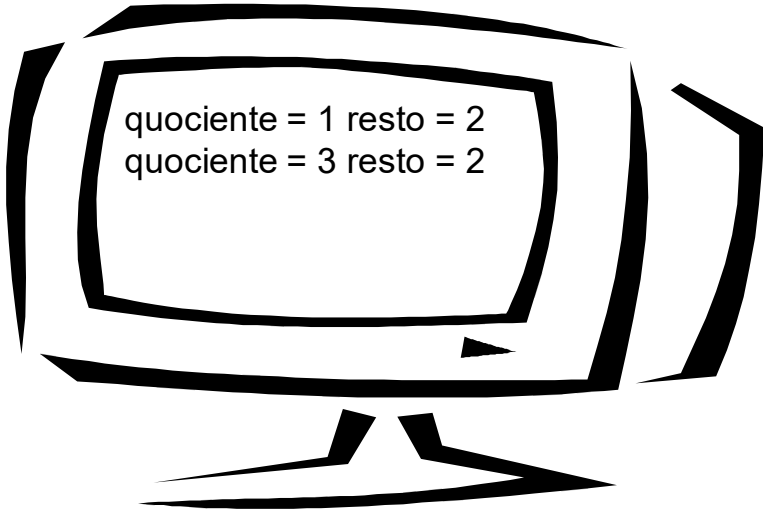
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << "quociente = " << quociente
       << " resto = " << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor
dividendo	14
divisor	4
quociente	3
resto	2



Memória	
Endereço	Valor
a	5
b	3

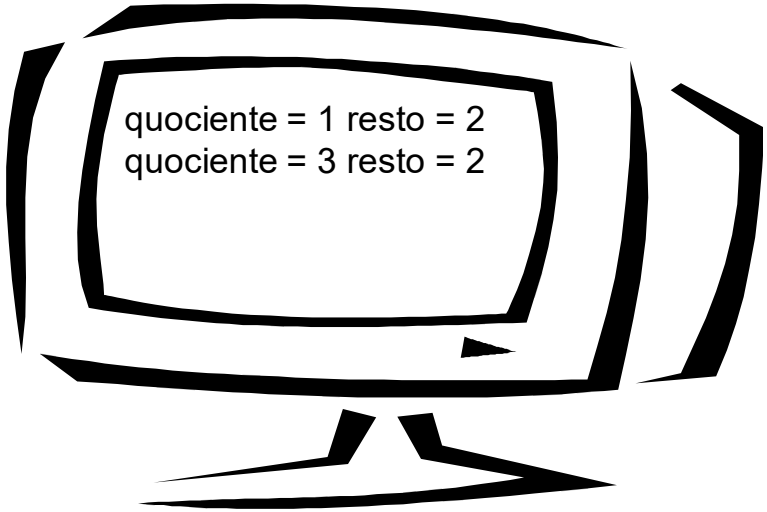
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor



Memória	
Endereço	Valor
a	5
b	3

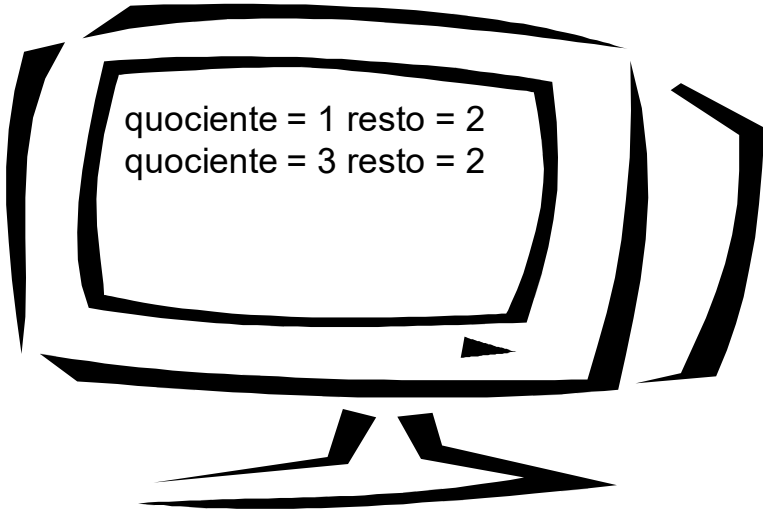
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor



Memória	
Endereço	Valor
a	5
b	3

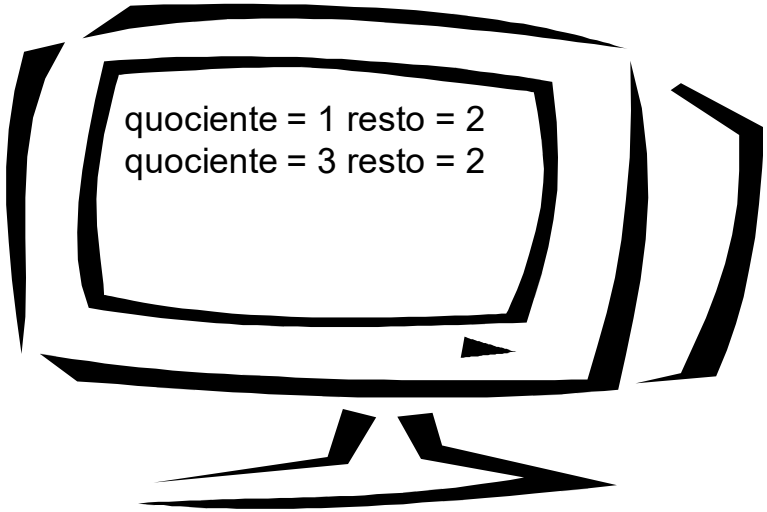
Procedimentos em C++

```
#include <iostream>
using namespace std;
//-----------------------------------------
void divide(int dividendo, int divisor)
{ int quociente,resto;

  quociente = dividendo / divisor;
  resto = dividendo - quociente * divisor;
  cout << ”quociente = “ << quociente
       << “ resto = ” << resto << endl;
}
//-----------------------------------------
int main()
{ int a=5,b=3;

  divide(a,b);
  divide(a*b-1,b+1);
  return 0;
}
```

Memória	
Endereço	Valor



Memória	
Endereço	Valor

Regras de Escopo de Identificadores

- Um identificador consiste em um nome de variável, tipo de dado, sub-algoritmo ou constante simbólica
- O **escopo** de um identificador é a região do programa na qual ele pode ser referenciado
  - Parâmetros e variáveis locais de um sub-algoritmo só podem ser referenciados diretamente dentro do próprio sub-algoritmo; nenhum outro sub-algoritmo pode fazer referência a eles
- Os parâmetros e as variáveis definidas em um sub-algoritmo são **variáveis locais**
  - Uma variável local é criada (alocada na memória) no momento em que o sub-algoritmo que a define é chamado
  - Uma variável local é liberada da memória no momento em que o sub-algoritmo que a define termina
  - O escopo de uma variável local é dentro do sub-algoritmo que a define
- Caso um mesmo identificador (nome de variável) seja declarado em sub-algoritmos distintos, esses identificadores são considerados distintos entre si (variáveis distintas)
- O uso de variáveis locais minimiza a ocorrência de “efeitos colaterais” em programação: o programador pode definir e utilizar as variáveis que desejar em um sub-algoritmo sem interferir com outros sub-algoritmos
- As variáveis definidas na “camada” mais externa de um programa são denominadas **globais** e têm sua existência durante toda a execução do programa
- O uso de variáveis globais deve ser evitado, pois elas podem ser alteradas por quaisquer sub-algoritmos

Resumo

- Nesta aula vimos os dois tipos de sub-algoritmos existentes: funções e procedimentos
- A função sempre retorna um valor no ponto onde foi chamada; já o procedimento pode retornar vários valores e seu retorno ao programa que chama é efetuado no comando subsequente
- Variáveis declaradas dentro de um sub-algoritmo (variáveis locais) têm sua existência somente quando o sub-algoritmo é executado e deixam de existir ao término da execução do sub-algoritmo
- Existem duas formas de correspondência entre parâmetros de definição e de chamada: por valor e por variável